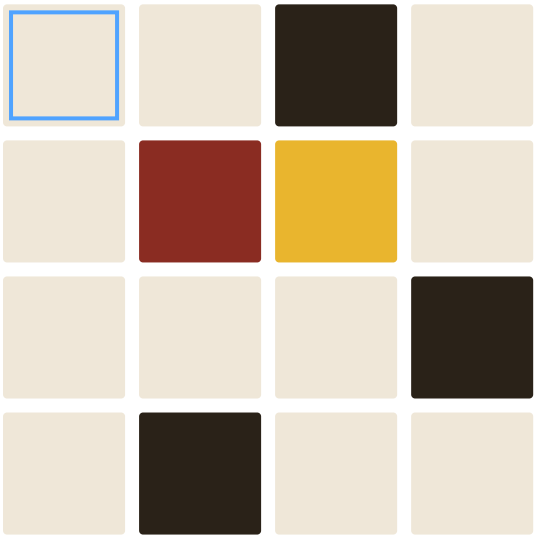
WUMPUS 3D · PROPOSTA EDUCACIONAL

Caverna do Pensamento

Como adaptar o Wumpus 3D para ensinar pensamento computacional a crianças e jovens de 10 a 18 anos, com ideias de jogos que já fazem isso.



Tipo	**Proposta de mudanças (nada foi alterado no repositório)**
Base	**Wumpus 3D, branch refatoracao (commit 14546c8)**
Data	**2 de novembro de 2026**
Público	**Professores, coordenação pedagógica e quem vai desenvolver**

Sumário

1 Resumo da proposta

2 Pensamento computacional: o que ensinar

3 O que o Wumpus 3D já tem e o que atrapalha

4 Jogos pesquisados e o que aproveitar

5 Os cinco modos em detalhe

6 Campanha de missões (exemplo)

7 Faixas etárias

8 Professor, avaliação e inclusão

9 Como isso entra no código atual

10 Roteiro sugerido

11 Referências

1 Resumo da proposta

O Wumpus 3D já é, por natureza, um **micromundo** no sentido de Papert (1980): um robô numa grade, que só sabe o que sente, precisa planejar passos, deduzir onde está o perigo e decidir sob incerteza. Isso cobre quase tudo o que o pensamento computacional pede. O que falta para crianças e jovens de 10 a 18 anos é **a porta de entrada**: hoje, para "programar" o jogo, é preciso escrever JavaScript, e a interface fala a língua de quem estuda IA na universidade.

A proposta é uma **versão educacional** ("Caverna do Pensamento", nome provisório) que mantém o motor do jogo e acrescenta cinco modos, em degraus:

MODO	PARA QUEM	IDEIA	INSPIRADO EM
1. Detetive	10+ anos	Joga com as próprias mãos e anota pistas num caderno no mapa; um robô-ajudante comenta as deduções.	Mundo de Wumpus do AIMA (Russell e Norvig (2020)); árbitro que já existe no jogo
2. Programador com blocos	10–15 anos	Monta o programa do robô com blocos (avançar, girar, repetir, se sentir…) numa campanha de missões curtas.	Lightbot, Blockly Games (Maze), Code.org/Minecraft, Karel
3. Cientista de agentes	14–18 anos	Passa dos blocos para o código (JavaScript ou Python) e coloca o agente para disputar a Arena da turma.	Blockly Games (Pond), CodeCombat, Human Resource Machine
4. Criador de cavernas	todas	Desenha mapas, cria desafios e troca com colegas por um código curto.	Scratch (criar e remixar)
5. Desplugado	todas	Tabuleiro e cartas impressos para jogar sem computador, em dupla: um "programa", o outro "é o robô".	Robot Turtles, CS Unplugged, Bebras

POR QUE ADAPTAR O WUMPUS EM VEZ DE COMEÇAR DO ZERO

- As regras já estão isoladas em `world.ts`, sem 3D nem tela: dá para trocar a interface sem tocar nelas.
- Os agentes já recebem só percepções e rodam isolados (Worker): um programa de blocos pode virar um agente comum, sem nenhum risco ao jogo.
- O árbitro (`referee.ts`) já mede "erros de lógica" e "riscos desnecessários": é avaliação formativa pronta.
- Sementes, desafios e relatórios já existem: o professor pode dar o mesmo conjunto de cavernas para a turma inteira e comparar.

O QUE ESTE DOCUMENTO NÃO FAZ

Nada foi alterado no repositório nem no site. É uma proposta: as ideias, as telas desenhadas e o roteiro servem para decidir o que construir.

2 Pensamento computacional: o que ensinar

Wing (2006) descreve o pensamento computacional como uma forma de resolver problemas que serve a todos, não só a quem programa. Brennan e Resnick (2012), estudando jovens no Scratch, propõem olhar para três dimensões, que este documento usa como guia:

DIMENSÃO	O QUE É	ONDE APARECE NO WUMPUS
Conceitos	Sequências, repetições, condicionais, eventos, dados, operadores, paralelismo.	Programa do robô (sequência de passos), "repetir até", "se sentir brisa", memória das salas visitadas.
Práticas	Experimentar e iterar, testar e depurar, reutilizar e remixar, abstrair e modularizar.	Executar passo a passo, ver onde o robô caiu, criar funções ("voltar para casa"), remixar mapas de colegas.
Perspectivas	Expressar-se, conectar-se com outros, questionar a tecnologia.	Criar cavernas, competir e colaborar na Arena, discutir "como uma máquina decide sem ver?".

Os desafios Bebras organizam suas tarefas em torno de abstração, decomposição, pensamento algorítmico, avaliação e generalização. No Wumpus, cada uma tem um momento natural:

HABILIDADE	MOMENTO NO JOGO
Decomposição	"Achar o ouro e voltar" vira: explorar com segurança → pegar → voltar pelo caminho conhecido.
Reconhecimento de padrões	Brisa sempre ao lado de poço; fedor ao lado do Wumpus; o mesmo trecho de programa serve em várias missões.
Abstração	O mapa 2D e o caderno de pistas ignoram o 3D e guardam só o que importa: visitado, seguro, suspeito.
Algoritmos	Regras do robô: "explore primeiro as salas seguras", "volte pelo caminho que conhece".
Avaliação	Comparar dois programas pelo número de blocos, de passos e de mortes nas mesmas cavernas.
Generalização	Um programa que vence uma caverna vence outras? A Arena responde com 50 cavernas sorteadas.

Alinhamento com a BNCC Computação

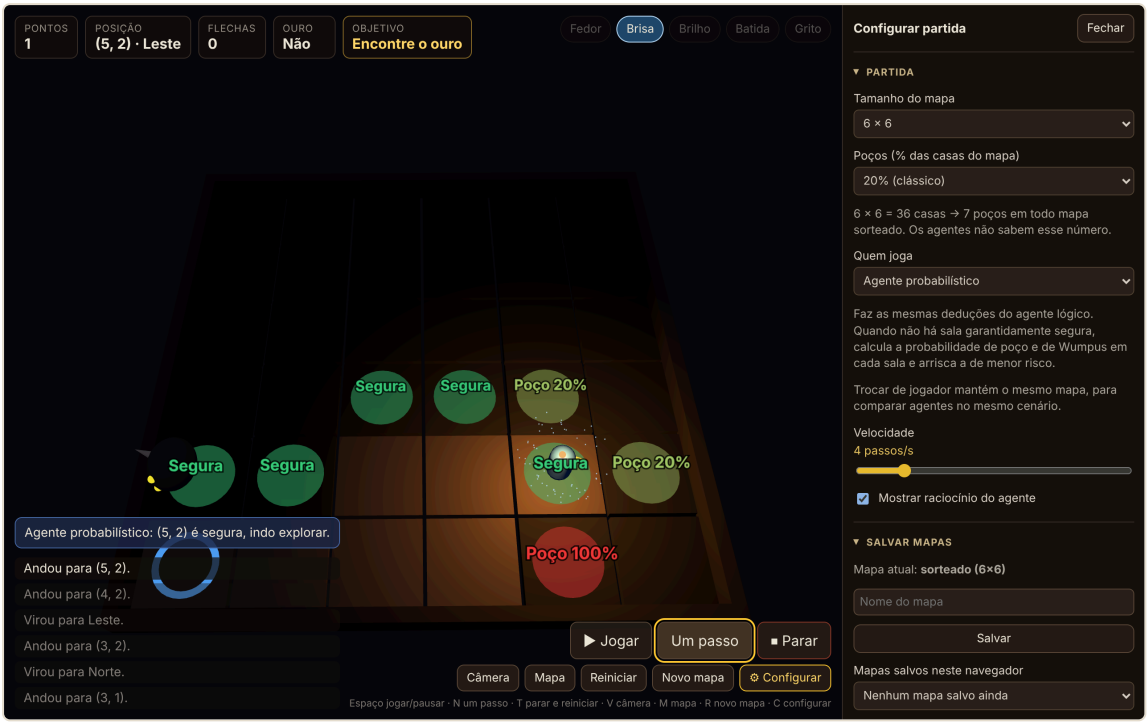
O Parecer CNE/CEB nº 2/2022 e a Resolução CNE/CEB nº 1/2022 (Brasil (2022)) complementam a BNCC com três eixos: **Pensamento Computacional**, **Mundo Digital** e **Cultura Digital**. A proposta toca nos três:

- **Pensamento Computacional**: núcleo de todos os modos (algoritmos, decomposição, abstração, depuração).
- **Mundo Digital**: como um agente "percebe" o ambiente por sensores e guarda uma representação dele; o que é um sorteio com semente.
- **Cultura Digital**: trabalho colaborativo, compartilhamento de mapas, ética de agentes que decidem sozinhos.

> **LIGAÇÃO COM INTELIGÊNCIA ARTIFICIAL**
>
> As "cinco grandes ideias" da iniciativa AI4K12, descritas por Touretzky et al. (2019), são percepção, representação e raciocínio, aprendizado, interação natural e impacto social. O Wumpus cobre muito bem as duas primeiras (o robô só sabe o que os sensores dizem e monta um modelo do mundo), e o agente genético dá um exemplo concreto de aprendizado para o ensino médio.

Recomenda-se que o professor confira, no texto oficial da BNCC Computação, os códigos das habilidades de cada ano antes de montar o plano de aula; este documento cita só os eixos.

3 O que o Wumpus 3D já tem e o que atrapalha



O jogo hoje: vista 3D, percepções no topo, registro de eventos e painel lateral.

Já ajuda

- **Grade e percepções simples**: fedor, brisa, brilho, batida e grito são "sensores" fáceis de explicar.
- **Crenças desenhadas no piso** ("Poço 56%", "Segura"): tornam visível o raciocínio do agente.
- **Pensamento do agente em texto** ("(2, 3) é segura, indo explorar").
- **Árbitro**: conta erros de lógica e riscos desnecessários de qualquer jogador, inclusive humano.
- **Sementes e desafios**: todos jogam as mesmas cavernas.
- **Relatórios e Análise**: gráficos prontos para comparar alunos e agentes.
- **Mapas em JSON**, salvar e importar.
- **Agentes isolados** num Worker, com contrato simples.

Atrapalha (para 10–18 anos)

- **Programar exige JavaScript** escrito à mão, e o contrato fala em `registerAgent`, `create(info)`.
- **Pontuação de adulto**: −1000 por morte, +500, −10 por tiro. Números grandes e negativos desanimam os menores.
- **Sem progressão**: o jogo começa no mapa completo, com Wumpus, poços e flecha ao mesmo tempo.
- **Linguagem técnica**: "probabilístico", "genoma", "semente", "fronteira".
- **Tema**: "O Wumpus te devorou!" funciona para adolescentes, mas pede um tom mais leve para os de 10–11 anos.
- **3D pesado** em computadores fracos de laboratório; percepções mostradas só por cor e texto pequeno.
- **Nada para o professor**: sem turmas, missões montadas ou plano de aula.

4 Jogos pesquisados e o que aproveitar

Foram analisados jogos e atividades usados no ensino de pensamento computacional. A coluna da direita diz o que cada um sugere para o Wumpus. Nenhum deve ser copiado: a ideia é aproveitar o **mecanismo pedagógico**, com arte, missões e textos próprios.

JOGO / ATIVIDADE	COMO FUNCIONA	IDEIA PARA O WUMPUS
Lightbot (SpriteBox; versão "Hour of Code" gratuita)	Programa-se um robô numa grade com fichas de comando; poucas casas para o programa obrigam a usar procedimentos e repetições.	**Limite de blocos por missão** e estrelas por programa curto; caixa "função" para "voltar para casa".
Blockly Games (Google; Maze, Bird, Turtle, Pond)	Sequência de jogos com blocos, cada um focando um conceito (laços, condicionais); o último (Pond) é uma disputa de "patos" programados, com blocos ou JavaScript.	Usar a **biblioteca Blockly** (código aberto) para o editor; transição **blocos → texto** como no Pond, que é exatamente a nossa Arena.
Code.org — Minecraft Adventurer	Tutoriais de uma hora com blocos num mundo conhecido; o fim é um nível livre.	Campanha de **~1 hora** para a primeira aula; nível livre ao final.
Karel the Robot Pattis (1981)	Robô literal num mundo em grade, ensinado por refinamento passo a passo.	O robô do Wumpus é um "Karel com sensores": reaproveitar a ideia de **refinamento** (criar comandos novos a partir dos simples).
Human Resource Machine (Tomorrow Corporation)	Começa com 2 comandos e chega a 11; cada fase premia programas curtos e rápidos.	**Liberar blocos aos poucos**; dois desafios de otimização por missão: menos blocos e menos passos.
CodeCombat	Jogo de aventura em que se digita Python ou JavaScript; painel do professor vê o código de cada aluno.	Modo texto com **Python** para o ensino médio e **painel da turma** com o código de cada um.
Scratch Resnick et al. (2009)	Criar e compartilhar histórias e jogos; remixar o projeto de outro.	**Editor de cavernas** e troca por código curto; remixar a caverna de um colega.
Robot Turtles (jogo de tabuleiro)	A criança dá cartas de comando e um adulto "executa" como se fosse o computador.	**Versão desplugada**: tabuleiro 4×4 impresso, cartas de comando e cartas de percepção.
CS Unplugged CS Unplugged	Atividades de computação com papel, cartões e movimento, sem computador.	Aula inicial sem tela: um aluno é o robô vendado, os colegas dão comandos e o "robô" só diz o que sente.
Desafio Bebras	Tarefas curtas de múltipla escolha, por faixa etária, sem exigir programação.	**Cartões-desafio** de 3 minutos ("onde está o poço?") para aquecimento e avaliação diagnóstica.
Machine Learning for Kids (Dale Lane)	Treinar modelos simples e usá-los como blocos no Scratch.	No ensino médio: "treine seu robô" com o **agente genético** já existente, vendo os pesos mudarem.

> **CUIDADO COM CÓPIAS**
>
> Inspiração em mecânicas é comum e saudável; usar nomes, personagens, fases ou arte de outros jogos não. Blockly é código aberto (licença Apache 2.0) e pode ser usado como biblioteca. Vale registrar que, segundo a Wikipédia, a manutenção do Blockly passou do Google para a Raspberry Pi Foundation a partir de novembro de 2025; conferir a situação antes de adotar.

5 Os cinco modos em detalhe

10–12 anos · Explorar

Detetive + desplugado
Blocos: sequência, repetição
Mapa 2D, sem números de pontuação

13–15 anos · Programar

Blocos: condicionais, funções, variáveis
Ver o código JS gerado
Criar e trocar mapas
Pontuação simplificada

16–18 anos · Criar agentes

Texto (JS/Python)
Agentes na Arena da turma
Probabilidade e IA
Algoritmo genético
Pontuação completa (AIMA)

Modo 1 — Detetive (10+ anos)

Caderno do Detetive

segura	poço?		
visitada	💨	poço?	

Minhas pistas

- Em (2,1) senti brisa.
- Em (1,1) não senti nada.

Minha conclusão

O poço está em (3,1) ou em (2,2).

Robô-ajudante:
Boa! E (1,2)? Ela é vizinha de (1,1), que não tinha brisa...

Esboço: o aluno marca no caderno o que sabe de cada sala; o robô-ajudante comenta com perguntas, sem entregar a resposta.

- O aluno joga com teclado, toque ou botões grandes, numa **vista 2D de cima** (a 3D vira opcional).
- Cada sala do **Caderno** recebe uma marca do aluno: visitada, segura, poço?, Wumpus?, poço certo. Isso torna a **abstração** explícita.
- O **robô-ajudante** usa a base de conhecimento que já existe (`knowledge.ts`) para comparar a marca do aluno com o que dá para deduzir, e responde com perguntas ("E a sala (1,2)? Ela é vizinha de uma sala sem brisa…"), nunca com a resposta pronta.
- O **árbitro** vira feedback amigável no fim: "Você entrou numa sala que dava para saber que era perigosa" em vez de "erro de lógica: 1".

Modo 2 — Programador com blocos (10–15 anos)



Esboço: paleta de blocos à esquerda, programa no meio (o bloco que está rodando fica destacado), caverna 2D à direita, com a percepção escrita em linguagem simples.

- **Blocos**: avançar, girar à esquerda/direita, pegar ouro, atirar; repetir N vezes, repetir até; se sentir brisa/fedor/brilho, senão; funções; e, mais tarde, "lembrar sala" (variáveis/listas).
- **Execução visível**: ▶ executar, ⏭ um passo, velocidade; o bloco atual brilha e o robô anda. Depurar fica concreto.
- **Campanha de missões** (capítulo 6), liberando blocos aos poucos (Human Resource Machine) e com limite de blocos (Lightbot).
- **Ver o código**: um botão mostra o JavaScript gerado pelos blocos, ponte para o modo 3.

Modo 3 — Cientista de agentes (14–18 anos)

- Editor de texto com **JavaScript** (o contrato atual) e, como opção, **Python** (por exemplo, rodando Python no navegador, como o Pyodide, dentro do mesmo Worker isolado).
- Modelo pronto com lacunas ("complete a regra de quando atirar"), em vez de arquivo vazio.
- **Arena da turma**: o professor cria uma competição com semente; cada aluno envia seu agente; o ranking e os relatórios já existem.
- **Trilha de IA** para 16–18 anos: ligar o desenho de crenças, mostrar a conta de probabilidade (o exemplo de 56% já está na documentação técnica) e treinar o agente genético.

Modo 4 — Criador de cavernas (todas as idades)

- Editor visual: clicar para pôr poço, Wumpus e ouro; o jogo avisa se a caverna não tem solução (a busca em largura já existe em `world.ts`).
- Compartilhar por **código curto** (o JSON de mapa já existe) e **remixar** a caverna de um colega.
- Desafio entre pares: "minha caverna derrota o seu programa?".

Modo 5 — Desplugado (todas as idades)

- PDF para imprimir: tabuleiro 4×4, fichas de poço, Wumpus e ouro, cartas de comando (avançar, girar, repetir, se…) e cartões de percepção.
- Dinâmica em dupla: o "programador" monta a sequência de cartas; o "robô" executa sem ver o mapa e só diz o que sente; o "mestre da caverna" conhece o mapa.
- Serve para a primeira aula, para escolas sem laboratório e para conversar sobre "o que é um algoritmo" antes da tela.

6 Campanha de missões (exemplo)

Uma sequência possível para o modo 2, pensada para 3 a 4 aulas de 50 minutos. Cada missão tem uma meta, os blocos liberados e o conceito em foco. As primeiras usam cavernas fixas e sem perigo, uma diferença importante em relação ao jogo atual, que sempre começa completo.

#	MISSÃO	META	BLOCOS	CONCEITO
1	Primeiros passos	Levar o robô até o ouro, sem perigos.	avançar, girar	Sequência
2	Volta para casa	Pegar o ouro e voltar à entrada.	+ pegar	Sequência, decomposição
3	O corredor longo	Mesmo caminho repetido muitas vezes; só cabem 4 blocos.	+ repetir N	Repetição
4	Escada	Padrão "avançar, girar, avançar, girar…".	(mesmos)	Padrões, repetição aninhada
5	Brisa!	Um poço no caminho; o robô sente brisa antes.	+ se sentir brisa	Condicional
6	Um programa, três cavernas	O mesmo programa precisa vencer 3 cavernas parecidas.	+ senão	Generalização, teste
7	Repetir até achar	O ouro está em lugar desconhecido.	+ repetir até	Laço com condição
8	Caminho de volta	Criar a função "voltar para casa" e usá-la em 2 missões.	+ função	Abstração, modularização
9	O monstro dorme	O Wumpus aparece (sem poços); fedor ao lado dele.	+ atirar	Condicional composta
10	Memória do robô	Não entrar duas vezes na mesma sala.	+ lembrar sala	Variáveis, listas
11	Caverna completa	Poços, Wumpus e ouro, como no jogo original.	todos	Integração
12	Arena da turma	Seu programa contra 20 cavernas sorteadas pela professora.	todos	Avaliação, generalização

Estrelas no lugar da pontuação

Para 10–15 anos, cada missão dá até **três estrelas**: ★ concluiu, ★★ sem morrer em nenhuma tentativa de teste da mesma missão, ★★★ com o menor número de blocos sugerido. A pontuação original (−1, −1000, +500…) continua existindo no modo 3 e na Arena, porque é ela que dá sentido ao agente genético.

> **MORRER NÃO REINICIA TUDO**
>
> No modo educacional, cair no poço mostra **onde** o programa falhou (o bloco fica vermelho) e volta o robô ao início da mesma caverna, com o programa intacto. Errar vira parte do ciclo de testar e depurar, uma das práticas de Brennan e Resnick.

7 Faixas etárias

	10–12 ANOS	13–15 ANOS	16–18 ANOS
Modos	Desplugado, Detetive, Blocos (missões 1–8)	Blocos (todas), Criador, primeiros passos no texto	Texto (JS/Python), Arena, trilha de IA
Conceitos	Sequência, repetição, condicional simples	Condicionais compostas, funções, variáveis, depuração	Busca em largura, lógica proposicional, probabilidade, otimização, algoritmo genético
Visual	Vista 2D, ícones grandes, tema leve ("o Wumpus te pegou")	2D com opção 3D	3D completo, crenças e números
Feedback	Estrelas, robô-ajudante com perguntas	Estrelas + quantidade de passos e blocos	Pontuação AIMA, relatórios, ranking
Duração	Atividades de 10–15 min	Missões de 15–20 min	Projetos de várias aulas

As faixas são indicativas. O que muda de verdade é o **degrau**: um aluno de 16 anos sem experiência começa nas missões; um de 12 que já programa em Scratch pode ir direto ao modo texto.

Linguagem e tema

- Textos curtos, em frases diretas, sem "fronteira", "genoma" ou "probabilístico" nos modos 1 e 2 (os termos aparecem num glossário com exemplos).
- Mensagens que encorajam ("Quase! O robô caiu no poço em (3,1). Que pista avisava isso?").
- Wumpus redesenhado com cara de criatura de desenho animado para os menores; a versão atual continua para o ensino médio.

8 Professor, avaliação e inclusão

Ferramentas do professor

- **Turma por código**: o professor cria um código; os alunos entram com um apelido. Nada de e-mail ou nome completo.
- **Missões da aula**: escolher missões, cavernas e semente; os alunos recebem o mesmo conjunto.
- **Painel**: quem concluiu o quê, quantas tentativas, número de blocos, onde mais se erra; abrir o programa de um aluno (ideia do CodeCombat).
- **Planos de aula** prontos por faixa etária, com o eixo da BNCC Computação de cada atividade.

Como avaliar

Brennan e Resnick (2012) sugerem combinar análise dos projetos, entrevistas sobre os artefatos e cenários de projeto. No Wumpus, isso pode ser:

INSTRUMENTO	COMO O JOGO AJUDA
Análise dos programas	Guardar cada versão do programa: dá para ver se o aluno passou a usar repetições e funções.
Entrevista sobre o artefato	Botão "explique seu programa": o aluno grava ou escreve por que cada bloco está ali.
Cenários de projeto	Mostrar o programa de um "colega fictício" com erro e pedir para achar e corrigir.
Diagnóstico rápido	Cartões-desafio no estilo Bebras no início e no fim da sequência de aulas.
Raciocínio lógico	O árbitro atual: entradas em salas que já dava para saber perigosas, riscos sem necessidade.

Acessibilidade

- Percepções com **ícone + palavra + som**, nunca só cor; paleta segura para daltonismo.
- Vista 2D navegável por teclado e leitor de tela ("Sala (2,1): você sente brisa"). A falta de descrição textual do 3D já está nas pendências do projeto.
- Botões grandes para toque; funciona em celular e Chromebook; modo leve sem 3D para computadores antigos.
- Leitura em voz alta das mensagens (síntese de voz do navegador).

Privacidade (LGPD)

O art. 14 da LGPD (Brasil (2018)) exige que o tratamento de dados de crianças e adolescentes atenda ao seu melhor interesse e, para crianças, consentimento específico de um dos pais ou responsável. A forma mais segura é **não coletar dados pessoais**: apelidos, progresso guardado no próprio navegador (como o jogo já faz) e exportação de arquivo para o professor. Se um dia houver servidor e contas, a escola e um especialista em proteção de dados devem participar do desenho.

9 Como isso entra no código atual

A arquitetura do projeto separa regras, 3D, agentes e interface; quase tudo da proposta pode ser **acrescentado**, sem mexer nas regras nem na Arena atuais.

PEÇA NOVA	ONDE	REAPROVEITA
Missões (cavernas fixas, blocos liberados, metas, estrelas)	`src/edu/missions.ts` (dados em JSON)	`MapLayout`, `parseLayout`, `WumpusWorld.fromLayout`
Editor de blocos	`src/edu/blocks.ts` com a biblioteca Blockly	Gerador de JavaScript no formato do contrato de agentes (`registerAgent`)
Execução passo a passo com destaque do bloco	`src/edu/runner.ts`	`WorkerSandbox` (o programa do aluno roda isolado), `playEpisode`
Vista 2D e caderno do detetive	`src/render/view2d.ts` (canvas ou SVG)	Lê o mesmo estado que `view.ts`; marcas do aluno comparadas com `Knowledge`
Robô-ajudante	`src/edu/helper.ts`	`Knowledge` (deduções) e `Referee` (erros de lógica)
Editor de cavernas	`src/ui/editor-panel.ts`	Verificação de caminho (`reachable`) e JSON de mapa
Turma, painel do professor	`src/edu/class.ts`	Sementes (`arenaMaps`), relatórios e tela de Análise
Python no navegador (opcional)	Worker separado	Mesmo contrato JSON de percepções e ações
Material desplugado	`scripts/docs/desplugado.py`	Gerador de PDF que o projeto já usa

> **INVARIANTES DO PROJETO QUE PEDEM DECISÃO**
>
> - **Pontuação (invariante 5)**: as estrelas devem ser uma camada *de exibição* do modo educacional. Mudar a pontuação do motor invalidaria o agente genético e os números medidos.
> - **Mapas sorteados (invariante 6)**: as primeiras missões precisam de cavernas sem poço ou sem Wumpus. Como mapas importados já não são checados, as missões podem ser mapas fixos em JSON, sem mudar o sorteio.
> - **Morte que não reinicia**: implementar como "recomeçar a mesma caverna mantendo o programa", na interface; a regra do motor continua igual.
> - **Agentes isolados (invariante 2)**: o programa de blocos deve rodar no Worker, como os agentes importados, e nunca receber o objeto do mundo.

Escolha técnica: um site ou dois?

Recomenda-se **o mesmo repositório e motor, com uma entrada separada** (por exemplo, `/escola/`), para que a versão educacional tenha tela inicial, linguagem e visual próprios sem esconder nada da versão atual, que continua servindo à universidade e à Arena dos desenvolvedores.

10 Roteiro sugerido

FASE	ENTREGA	PARA VALIDAR	TAMANHO
0. Desplugado	PDF de tabuleiro e cartas; roteiro de uma aula.	Uma turma de 10–12 anos joga sem computador.	Pequeno
1. Base	Entrada `/escola/`, vista 2D, textos simples, percepções com ícone e som.	Crianças entendem as percepções sem ajuda?	Médio
2. Blocos	Editor Blockly, execução passo a passo, missões 1–8, estrelas.	Teste com 2–3 turmas: onde travam? quanto tempo por missão?	Grande
3. Detetive	Caderno de pistas e robô-ajudante.	As marcas dos alunos melhoram ao longo das cavernas?	Médio
4. Professor	Turma por código, painel, exportar progresso, planos de aula.	Professores conseguem montar uma aula sozinhos?	Médio
5. Criar e competir	Editor de cavernas, missões 9–12, modo texto e Arena da turma.	Alunos do ensino médio criam agentes que vencem o lógico?	Grande
6. IA e Python	Trilha de probabilidade e genético; Python opcional.	Uso em disciplina eletiva do ensino médio.	Médio

"Tamanho" é relativo entre as fases. Os prazos dependem de quem vai desenvolver e de quantos testes com turmas forem possíveis; recomenda-se testar com alunos ao fim de cada fase antes de seguir.

Riscos

RISCO	COMO REDUZIR
Carga cognitiva alta (3D + regras + programação ao mesmo tempo)	Começar desplugado e em 2D; um conceito novo por missão.
Tema assustador para os menores	Arte e mensagens mais leves no modo 10–12; tema original no ensino médio.
Computadores fracos ou sem internet	Vista 2D leve; o jogo já funciona offline depois de carregado e salva no navegador.
Professores sem formação em computação	Planos de aula prontos, vídeos curtos e o modo desplugado.
Dados de menores	Sem contas nem dados pessoais; tudo local; exportação por arquivo.
Copiar jogos existentes	Usar só mecânicas pedagógicas gerais; arte, missões e textos próprios.

11 Referências

Obras e documentos

1. BEBRAS — International Challenge on Informatics and Computational Thinking (criado na Lituânia por Valentina Dagienė; no Brasil, organizado pela UpMat Educacional). bebras.org.
2. BRASIL. *Lei nº 13.709, de 14 de agosto de 2018* — Lei Geral de Proteção de Dados Pessoais (LGPD), art. 14 (dados de crianças e adolescentes).
3. BRASIL. Conselho Nacional de Educação. *Parecer CNE/CEB nº 2/2022* (BNCC Computação — Complemento à BNCC), aprovado em 17 de fevereiro de 2022, homologado em 3 de outubro de 2022; e *Resolução CNE/CEB nº 1, de 4 de outubro de 2022*. Disponível em: gov.br/mec/pt-br/cne.
4. BRENNAN, Karen; RESNICK, Mitchel. New frameworks for studying and assessing the development of computational thinking. In: *Annual Meeting of the American Educational Research Association (AERA)*, Vancouver, 2012. Disponível em: scratched.gse.harvard.edu/ct/files/AERA2012.pdf.
5. CS UNPLUGGED. Universidade de Canterbury (Nova Zelândia). Atividades de computação sem computador. csunplugged.org.
6. PAPERT, Seymour. *Mindstorms: Children, Computers, and Powerful Ideas*. New York: Basic Books, 1980. Texto disponibilizado pelo MIT Media Lab (media.mit.edu/publications/mindstorms).
7. PATTIS, Richard E. *Karel the Robot: A Gentle Introduction to the Art of Programming*. New York: John Wiley & Sons, 1981.
8. RESNICK, Mitchel et al. Scratch: programming for all. *Communications of the ACM*, v. 52, n. 11, p. 60–67, 2009. DOI: 10.1145/1592761.1592779.
9. RUSSELL, Stuart; NORVIG, Peter. *Artificial Intelligence: A Modern Approach*. 4. ed. Hoboken: Pearson, 2020. (Mundo de Wumpus: cap. 7 e seção 12.7.)
10. TOURETZKY, David; GARDNER-McCUNE, Christina; MARTIN, Fred; SEEHORN, Deborah. Envisioning AI for K-12: What should every child know about AI? In: *Proceedings of the AAAI Conference on Artificial Intelligence*, v. 33, p. 9795–9799, 2019. DOI: 10.1609/aaai.v33i01.33019795.
11. WING, Jeannette M. Computational thinking. *Communications of the ACM*, v. 49, n. 3, p. 33–35, 2006. DOI: 10.1145/1118178.1118215. Cópia da autora: cs.columbia.edu/~wing/publications/Wing06.pdf.

Páginas consultadas sobre os jogos (novembro de 2026)

- Lightbot — recursos para educadores: lightbot.com/resources.html
- Blockly Games — sobre: blockly.games/about
- Blockly (biblioteca): developers.google.com/blockly; Wikipédia, verbete "Blockly" (transição para a Raspberry Pi Foundation)
- Code.org — Minecraft Adventurer: studio.code.org/s/mc
- Human Resource Machine — Wikipédia (en.wikipedia.org/wiki/Human_Resource_Machine) e página da Tomorrow Corporation no itch.io
- CodeCombat — análise da Common Sense Education: commonsense.org/education/reviews/codecombat
- Robot Turtles — reportagens da NPR (2013) e TechCrunch (2013)
- Bebras no Brasil — reportagens de Folha de Pernambuco e Catraca Livre sobre o desafio organizado pela UpMat Educacional
- Bebras — faixas etárias: bebras.uk e materiais da TU Wien
- Machine Learning for Kids — Dale Lane, No Starch Press, 2021 (penguinrandomhouse.com)
- AI4K12 — resumo das cinco grandes ideias: raspberrypi.org/blog/ai-education-ai4k12-big-ideas-ai-thinking

Os dados de cada jogo (comandos, conceitos, faixas etárias) vêm dessas páginas e podem mudar. Preços e disponibilidade não foram considerados.